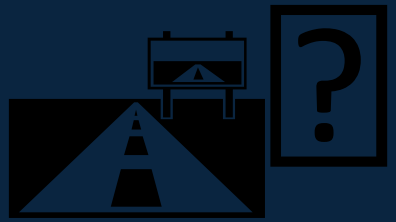
Université Paris Cité
Ecole d'ingénieurs Denis Diderot

Analyseur Intelligent

de Scènes Autoroutières

Module Intelligence Artificielle 2025–2026 | Scénario : Autoroute

Presenté par:
- Mahmoud Aziz Ben Khalifa
- Zakaria Belbakkal
- Dion Rexhepi



Sommaire du Projet

01 **Contexte & Objectifs**
Dataset BDD100K, 7 classes, scénario autoroute

02 **Architecture Globale**
Pipeline YOLO → Agent LLM → Streamlit

03 **Outils & Environnement**
Anaconda, Jupyter, PyCharm, Kaggle GPU

04 **Module A — Données & YOLO**
Prétraitement, conversion, hyperparamètres

05 **Architecture YOLO**
CSPNet Backbone, FPN Neck, Head, NMS

06 **Module B — Agent LLM**
LLaMA 3.3, 4 outils, boucle ReAct, JSON

07 **Architecture Agent LLM**
Séquence d'appels, system prompt, outils détaillés

08 **Module C — Streamlit**
Dashboard, 3 onglets, visualisations

09 **Résultats YOLOv11s**
Courbes, matrices, bounding boxes, analyse

10 **Résultats YOLOv8s**
Courbes, matrices, bounding boxes, analyse

11 **Comparaison & Analyse**
v8s vs v11s, toutes métriques

12 **Conclusion**
Bilan, perspectives

Contexte & Objectifs

Problématique

Analyser automatiquement des images de dashcam autoroutières pour détecter les objets (voitures, camions, piétons…) et évaluer le niveau de risque routier en temps réel grâce à un agent IA.

Dataset — BDD100K (scene=highway)

17 414 images train	**2 499** images val
7 classes	**1280×720** résolution

Environnement & Outils de Développement

Anaconda

Gestionnaire d'environnements

```
conda activate tensorflow
Isolement des dépendances Python
Gestion packages conda + pip
```

Jupyter Notebook

Prototypage interactif

```
Notebook 00_Filter_Highway.ipynb
Exploration dataset BDD100K
Visualisation inline des résultats
```

PyCharm

IDE développement principal

```
agent_llm.py — Agent LLM
app.py — Interface Streamlit
Débogage pas à pas avancé
```

Kaggle

GPU cloud (T4 / P100)

```
Notebook Yolov8_train.ipynb
Dataset BDD100K Highway
GPU gratuit pour entraînement
```

Ultralytics YOLO

Framework détection objet

```
YOLOv8s + YOLOv11s
ultralytics==8.3.0
API unifiée train/val/predict
```

Groq API

Inférence LLM ultra-rapide

```
LLaMA 3.3 70B Versatile
< 2s par analyse complète
Groq==0.9.0 (client Python)
```

Architecture Globale du Pipeline

Image Dashcam
Upload

- imgsz: 1280px
- 7 classes
- BDD100K

→

Module A YOLOv11s
Détection

- CSPNet backbone
- FPN+PAN neck
- NMS conf≥0.35

→

Module B Agent LLM
Analyse

- LLaMA 3.3 70B
- 4 tools ReAct
- JSON structuré

→

Module C Streamlit
Dashboard

- Jauge risque
- Bbox annotées
- Rapport JSON

YOLOv11s

ENTRÉE

Image dashcam

1280 × 720 pixels
Format JPG/PNG
RGB 3 canaux

Normalisation

Valeurs pixel [0, 1]

Paramètres

conf_threshold = 0.35
IoU_threshold = 0.45
imgsz = 1280

TRAITEMENT

Backbone CSPNet

Extrait les features
de l'image (Conv + C2f)
Stride 8 / 16 / 32

Neck FPN + PAN

Fusionne les features
à plusieurs echelles
(petits & grands objets)

Head Detection

Predit classe + bbox
pour chaque objet
(anchor-free, DFL)

SORTIE

Bounding Boxes

Coordonnees (x,y,w,h)
pour chaque detection
normalisees [0,1]

Classes

car · truck · bus
pedestrian · rider
traffic sign · light

Scores confiance

Probabilite 0→1
Ex: car 0.91
Ex: truck 0.74

Module A — Pipeline : Dataset & Entraînement

Pipeline de préparation du dataset

C1 BDD100K Brut
- 100k images
- Tous scenarios
- Labels JSON

→

C2 Filtre highway
- scene=highway
- 17 414 train
- 2 499 val

→

C3 JSON YOLO .txt
- box2d-(cx,cy)
- Normalise [0,1]
- 1 .txt/image

→

C4 YAML Config
- nc: 7 classes
- chemins train/val
- autoroute.yaml

→

C5 Train GPU
- Kaggle T4/P100
- Serveur univ.
- ultralytics

Conversion Labels BDD100K → Format YOLO

Entree BDD100K JSON

```
{
  "frames": [{
    "objects": [{
      "category": "car",
      "box2d": {
        "x1":452, "y1":210,
        "x2":698, "y2":390
      }
    }]
  }]
}
```

→

```
cx=(x1+x2)/(2W)
cy=(y1+y2)/(2H)
w=(x2-x1)/W
h=(y2-y1)/H
```

Sortie YOLO .txt

```
# class  cx      cy      w       h
0  0.4336 0.4167 0.1922 0.2500

# 0 = car
# Valeurs [0,1]
# 1 ligne = 1 objet
# 1 fichier .txt = 1 image
# clip(valeur, 0, 1)
```

Cellule 4 — dataset.yaml

```
path: /kaggle/working/dataset
train: images/train
val:   images/val

nc: 7
names:   0: car
         1: truck
         2: bus
         3: pedestrian
         4: rider
         5: traffic sign
         6: traffic light
```

Module A — Données & Hyperparamètres d'Entraînement

YOLOv11s

epochs :	30
imgsz :	1280 px
batch :	8
device :	GPU cuda:0
hsv_h / hsv_s / hsv_v :	0.015 / 0.7 / 0.4
fliplr / mosaic :	0.5 / 1.0
translate / scale :	0.1 / 0.5
degrees :	0.0 (dashcam fixe)
save_period :	5
optimizer :	SGD

YOLOv8s — Kaggle GPU T4/P100

epochs :	80
imgsz :	640 px
batch :	16
optimizer :	AdamW
lr0 / lrf :	0.001 / 0.1
momentum :	0.937
weight_decay :	0.0005
warmup_epochs / cos_lr :	3 / True
mosaic / mixup :	0.7 / 0.1
close_mosaic / patience :	10 / 20

Module B — Logique des 4 Outils de l'Agent LLM

HighwaySceneAgent — LLaMA 3.3 70B via Groq — tool_choice=auto — boucle ReAct ≤ 8 appels

W 01 get_weather_conditions

Appele si position GPS fournie

Logique :

Interroge OpenWeatherMap avec lat/lon. Convertit code meteo en etat de route. Sans cle API : simule conditions seches.

Exemple :

```
lat=48.8, lon=2.3
-> condition: pluie
-> road: mouille
-> coeff x1.5
```

Adapte vitesse (110km/h si pluie) et distance securite x1.5

D 02 calculate_safety_distance

Appele apres get_weather (road_condition connu)

Logique :

R412-12 : dist=(v/3.6)x2xcoeff. sec=1.0, mouille=1.5, brouillard=2.0, verglas=3.0. PL x1.3.

Exemple :

```
speed=120, mouille
-> (120/3.6)x2x1.5
-> 100m recommande
-> 120m legal min
```

Produit distance_securite_conseillee dans le JSON final

R 03 get_highway_rules

Appele si situation speciale : camion proche, pluie, urgence...

Logique :

Base interne Python (dict). 7 situations. Retourne regles[] + article de loi.

Exemple :

```
situation=poids_lourds
-> PL limite 110km/h
-> Angle mort 40m
-> article R413-2
```

Enrichit recommandations avec cadre legal applicable

G 04 estimate_object_distance

Appele pour chaque objet proche (bbox_height > 20% image)

Logique :

Triangulation : dist=(H_reelle x focal)/H_px. car=1.5m, truck=3.8m. focal=1050x(H/720). Precision +-20%.

Exemple :

```
truck, bbox_h=180px
-> focal=1050
-> dist=3.8x1050/180
-> 22.2m : CRITIQUE
```

Impact : Alerte CRITIQUE si truck<15m, ELEVE si <40m

Agent LLM (LLaMA 3.3)

ENTRÉE

Detections YOLO

Liste d'objets detectes
bbox + classe + score

Contexte conduite

Vitesse estimee: 120km/h
Position GPS (opt.)
Heure de la journee

System Prompt

Role: expert securite
Niveaux: FAIBLE→CRITIQUE
Format: JSON strict

TRAITEMENT

Analyse de scene

LLM lit les detections
et evalue le danger
(densite, proximite)

Appel des 4 outils

Meteo, distance securite
regles code de la route
distance geometrique

Raisonnement

Synthese des resultats
outils → niveau risque
→ recommandations

SORTIE

Niveau de risque

FAIBLE / MOYEN
ELEVE / CRITIQUE
+ score 0 → 100

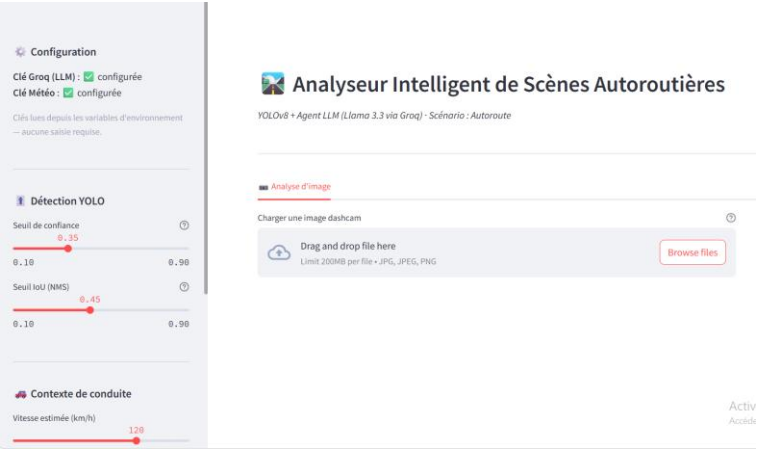
Recommandations

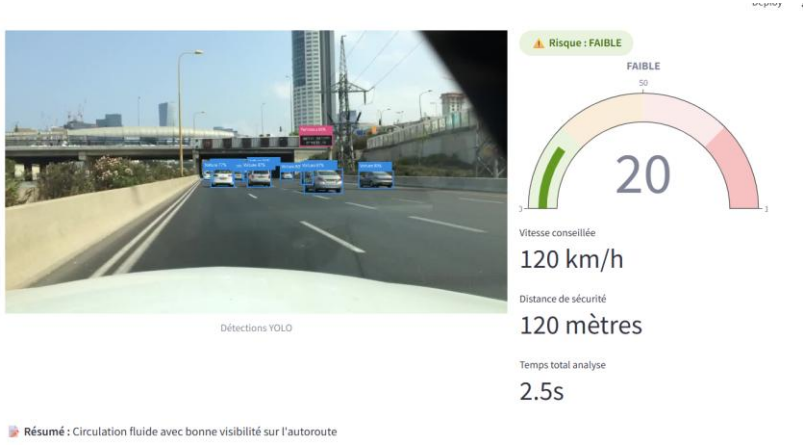
Vitesse conseillee
Distance de securite
Actions immediates

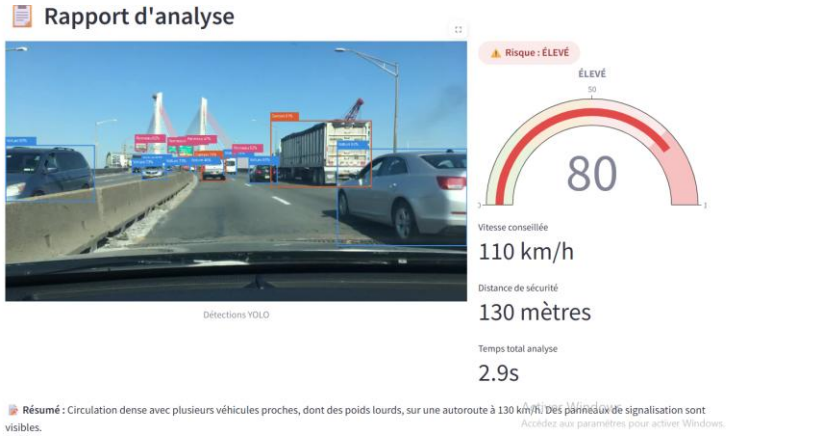
Rapport JSON

Resume de scene
Facteurs de risque
Metadonnees (timing)

Module C — Interface Streamlit Dashboard



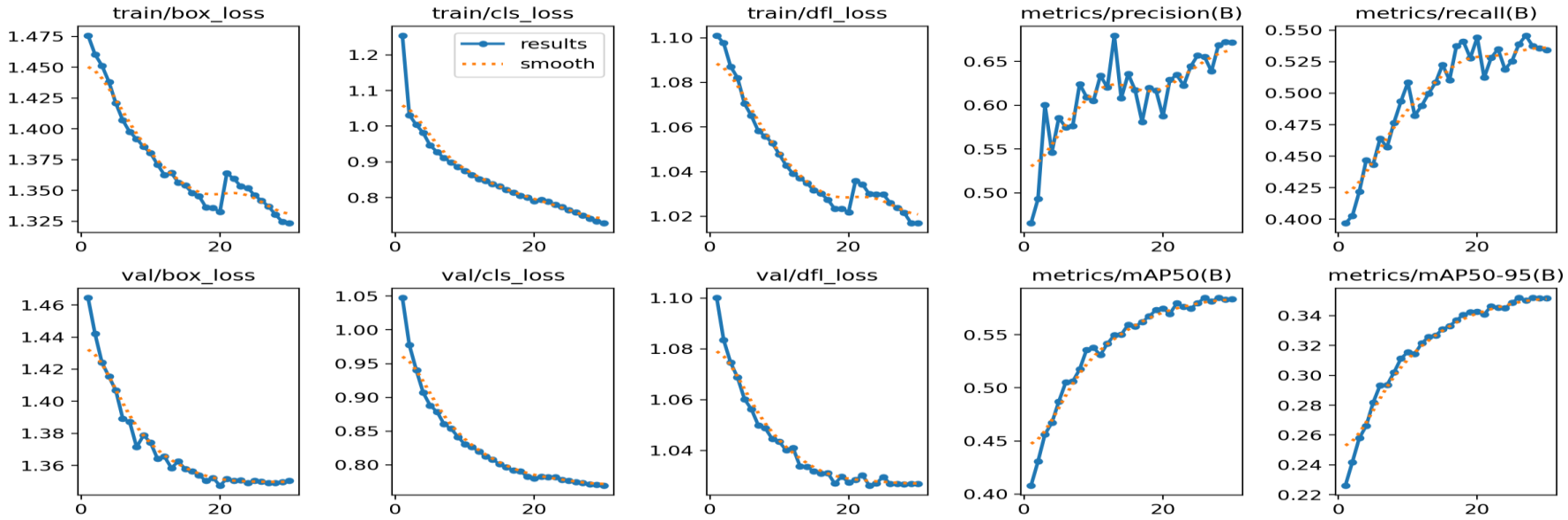




Résultats YOLOv11s — Courbes d'Entraînement Officielles

58.5%	35.2%	67.1%	53.4%
mAP@0.5	mAP@0.5:0.95	Précision	Rappel

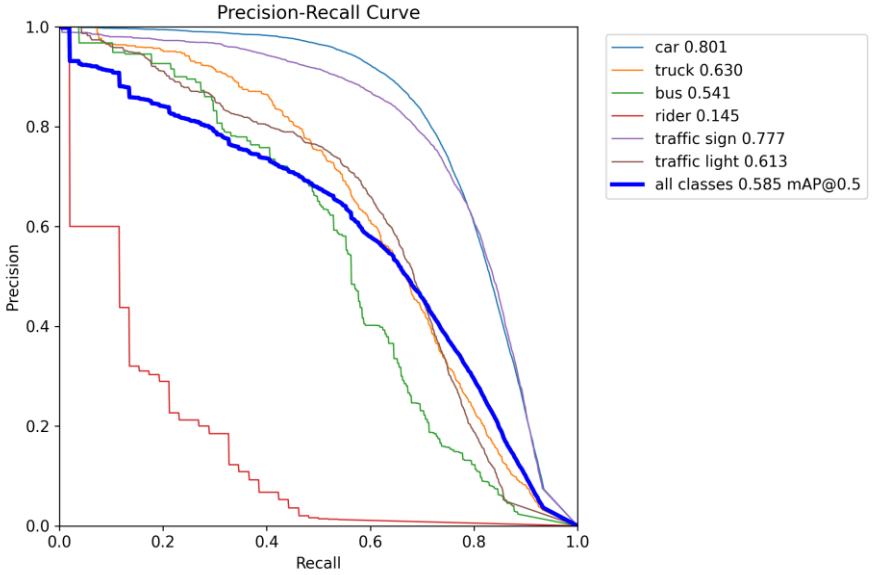
Courbes officielles Ultralytics (30 époques)



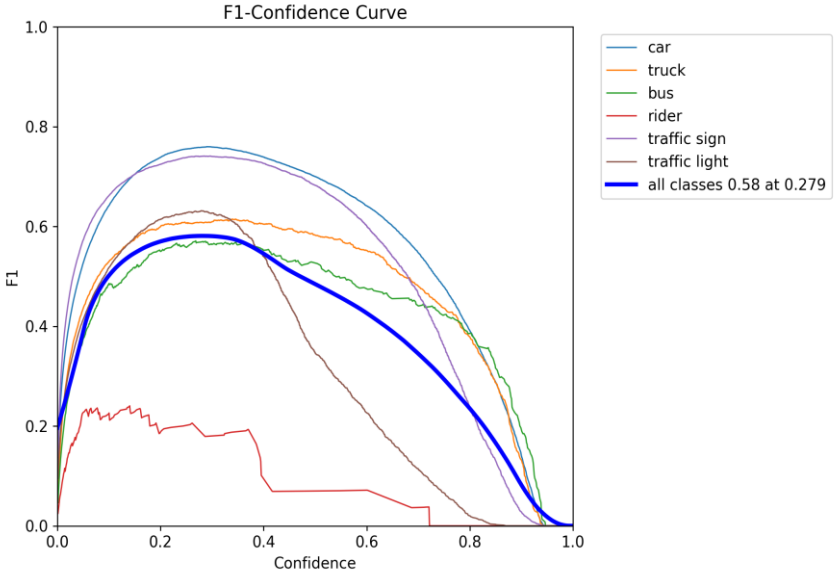
train/box_loss · train/cls_loss · train/dfl_loss · precision · recall · val/box_loss · val/cls_loss · val/dfl_loss · mAP@0.5 · mAP@0.5:0.95

YOLOv11s — Courbes PR & F1 par Classe

Precision-Recall Curve



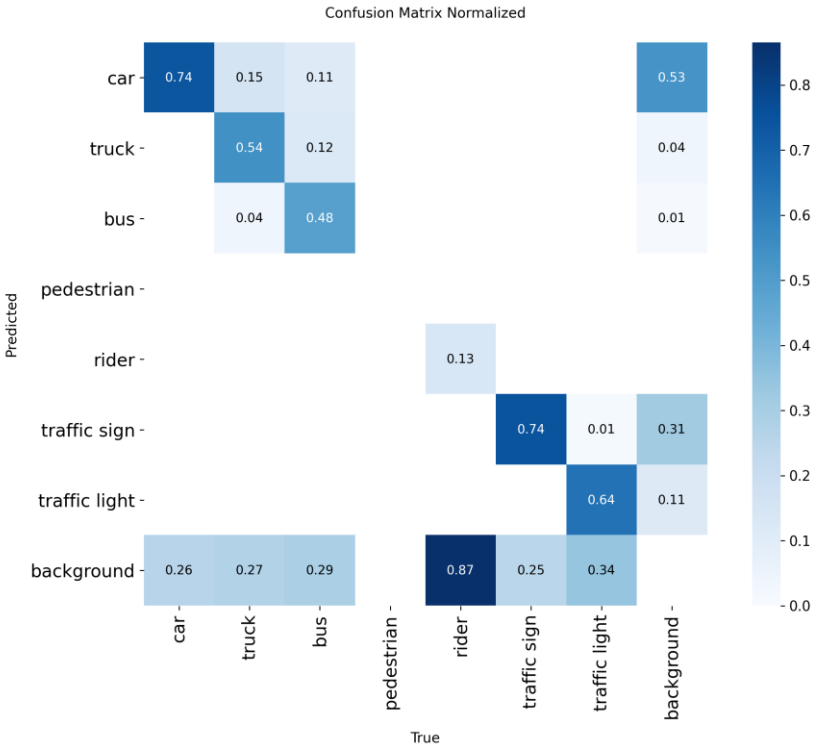
F1-Confidence Curve



Analyse des courbes YOLOv11s

PR : La classe car obtient la meilleure AP (0.801) grâce à sa forte représentation. Le rider affiche AP=0.145 (classe rare, sous-représentée sur autoroute). F1 optimal à conf≈0.279 (toutes classes) — seuil de confiance optimal pour l'application.

YOLOv11s — Matrice de Confusion Normalisée

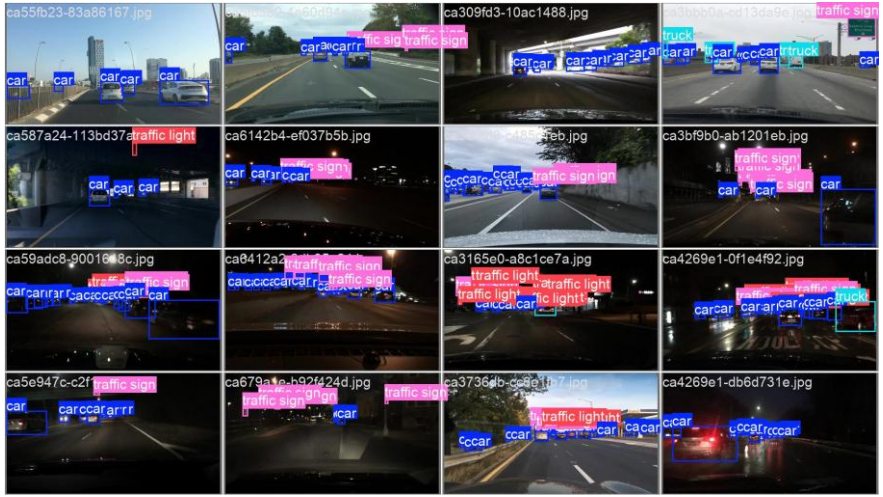


Analyse par classe

car	0.74	Bonne détection. 15% confondu avec background, 11% manqué.
truck	0.54	Moyen. Confusion car/truck fréquente (gabarits similaires de loin).
bus	0.48	Difficile. Ressemble à truck sur autoroute.
traffic sign	0.74	Bon. 31% classés background (petite taille, loin).
traffic light	0.64	Correct. 11% classé background.
rider	N/A	Non détecté. Trop rare dans le dataset highway.

YOLOv11s — Détections sur Images de Validation

Ground Truth (labels)



Prédictions YOLOv11s

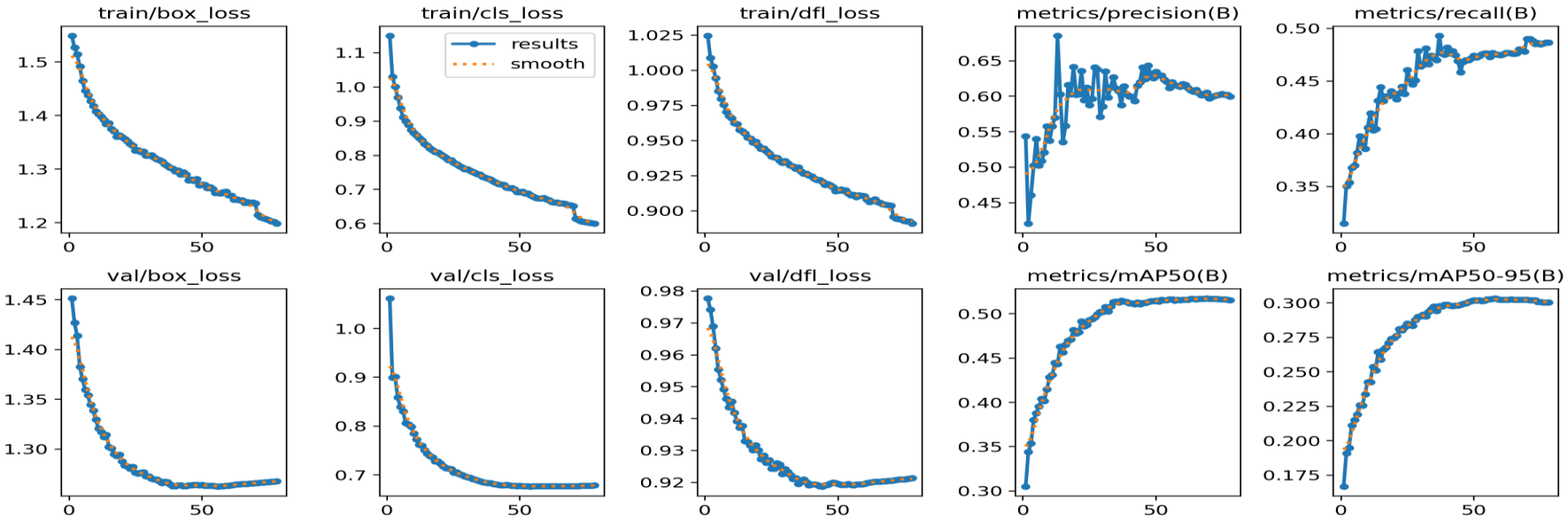
Observations sur les Prédictions

- ✓ Bonne détection des voitures de jour et de nuit (conf ≥ 0.6–0.9)
- ✓ Détection correcte des traffic signs (roses) même à distance modérée
- ⚠ Quelques double-détections (NMS à conf=0.35) sur les panneaux lointains
- ⚠ Images nocturnes : légère baisse de confiance (0.3–0.4) — lumière artificielle
- ℹ imgsz=1280 vs 640 : meilleure résolution → petits objets mieux détectés

Résultats YOLOv8s — Courbes d'Entraînement Officielles

51.5%	30.0%	60.0%	48.7%
mAP@0.5	mAP@0.5:0.95	Précision	Rappel

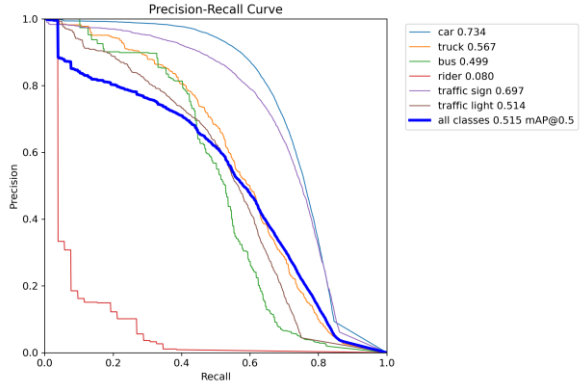
Courbes officielles Ultralytics (78 époques sur Kaggle)



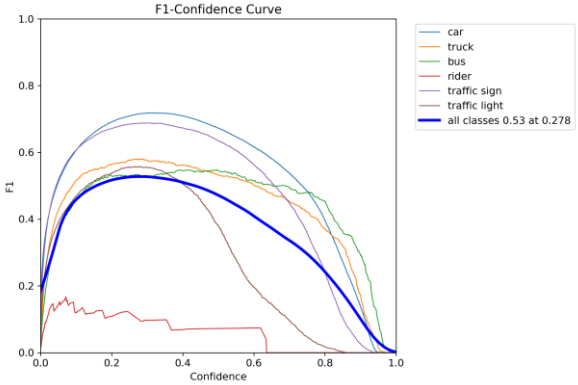
train/box_loss · train/cls_loss · train/dfl_loss · precision · recall · val losses · mAP@0.5 · mAP@0.5:0.95

YOLOv8s — Courbes PR, F1 & Matrice de Confusion

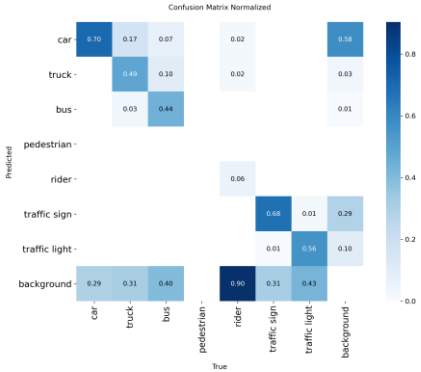
PR Curve



F1-Confidence Curve



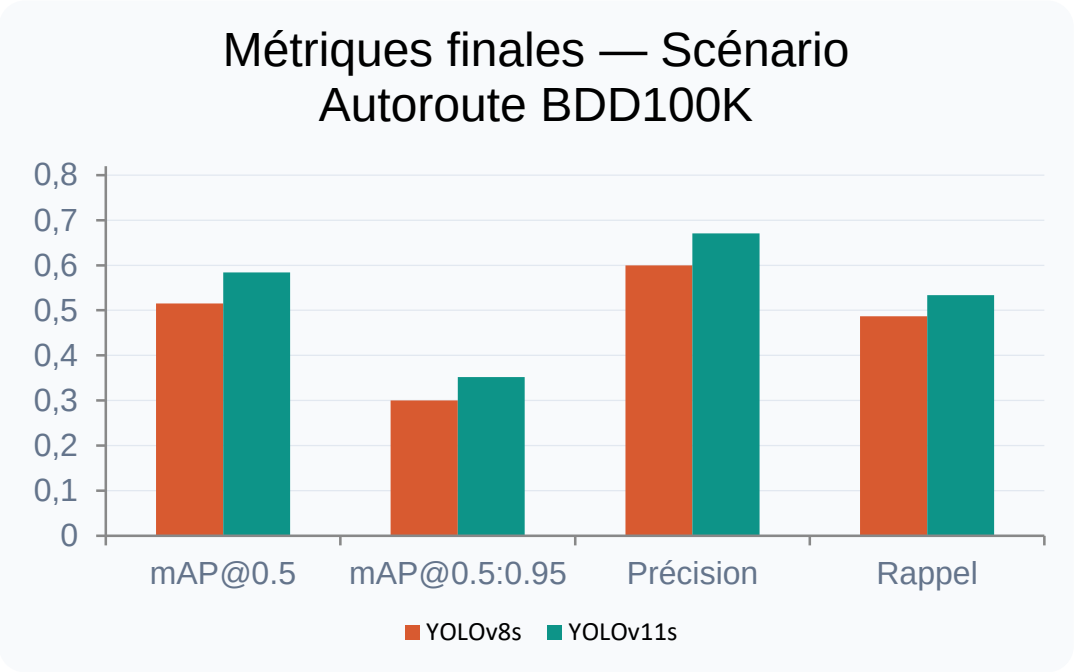
Matrice Confusion Normalisée



Ground Truth

Prédictions YOLOv8s

Comparaison Complète — YOLOv8s vs YOLOv11s



Critère	YOLOv8s	YOLOv11s	Delta
mAP@0.5	0.515	0.584 ✓	+13.4%
mAP@0.5:0.95	0.300	0.352 ✓	+17.3%
Précision	0.600	0.671 ✓	+11.8%
Rappel	0.487	0.534 ✓	+9.6%
car AP	0.734	0.801 ✓	+9.1%
truck AP	0.567	0.630 ✓	+11.1%
traffic sign AP	0.697	0.777 ✓	+11.5%
Époques	78	30 ✓	−62%
imgsz	640 px	1280 px	×2

✓ Conclusion : YOLOv11s est supérieur sur toutes les métriques en 3× moins d'époques. L'imgsz=1280 améliore la détection des petits objets lointains (voitures, panneaux). Le serveur universitaire offre plus de VRAM que Kaggle T4, permettant batch=8 avec imgsz=1280.

Conclusion & Bilan

Pipeline complet & opérationnel

YOLO → Agent LLM → Streamlit — bout en bout fonctionnel

YOLOv11s supérieur

mAP@0.5=58.5% en seulement 30 époques

Agent LLM pertinent & rapide

4 outils, code de la route français, réponse JSON < 2s

Maîtrise des outils IA

Anaconda, Jupyter, PyCharm, Kaggle GPU, Groq API